

GitHub Trending Top 10

每日热门开源项目深度解读

2026-10-30

今日榜单

1 **thedotmack/claude-mem** TypeScript
Persistent Context Across Sessions for Every Agent – Captures everything your ag... 2天

2 **milind-soni/OpenMausBot** TypeScript
Open-source Grok Bot alternative with a virtual machine that bots can use 2天

3 **pbakaus/impeccable** Rust
The design vocabulary for your coding agent. Steer with polish and bolder, revie... 7天

4 **CopilotKit/OpenBot** TypeScript
Open-source AI coworkers that each get a computer of their own: a browser, files... 2天

5 **Renhuai123/ziwei-doushu** TypeScript
紫微斗数开源排盘引擎 — 基于倪海厦《天纪》体系，含完整排盘算法、四化系统、格局知识… NEW

6 **expo/skills** Shell
A collection of AI agent skills for working with Expo projects and Expo Applicat... NEW

7 **macro-inc/macro** Rust
Macro is a unified workspace for teams: email, chat, docs, tasks, agents, calls,... NEW

8 **mattpocock/skills** Shell
Skills for Real Engineers. Straight from my .agents directory. NEW

9 **pingdotgg/t3code** TypeScript
NEW

10 **HKUDS/DeepTutor** Python
DeepTutor: Lifelong Personalized Tutoring.https://deeptutor.info/. 4天

#1 TypeScript 连续 2 天

claude-mem

Persistent Context Across Sessions for Every Agent – Captures everything your agent does during sessions, compresses it with AI, and injects relevant context back into future sessions. Works with Claude Code, OpenClaw, Codex, Gemini, Hermes, Copilot, OpenCode + More

Stars **105,707** Forks **9,314** Today **+1,145**

https://github.com/thedotmack/claude-mem

项目简介：claude-mem 是一个为 AI 编程代理（Agent）设计的持久化记忆压缩系统，核心解决的是"会话失忆"问题——每次新开对话，Agent 都不记得之前做过什么。它通过捕获会话全过程、用 AI 压缩成结构化记忆，并在后续会话中注入相关上下文，让 Agent 具备跨会话的连续工作能力。

核心功能： 1. **会话捕获**：自动记录 Agent 在会话中的所有操作（工具调用、决策、文件变更等），无需手动干预。 2. **AI 压缩**：将冗长的原始会话记录压缩为精炼的记忆片段，控制上下文体积。 3. **上下文注入**：在新会话开始时，检索并注入与当前任务相关的历史记忆，实现"续接"。 4. **多 Agent 兼容**：支持 Claude Code、Codex、Gemini、Copilot、OpenCode 等多种编程 Agent，不绑定单一生态。

适用场景：适合长期使用 AI 编程代理的开发者，尤其是跨多天推进同一项目、需要 Agent 记住架构决策和踩坑经验的场景；也适合团队希望让 Agent 积累"项目知识"而非每次从零开始的情况。

技术亮点：TypeScript 实现、Node ≥20，核心难点在于"压缩什么、注入什么"的检索与摘要策略——既要避免上下文膨胀，又要保证相关性。多 Agent 适配层设计使其具备跨工具的可移植性，这也是它能在短时间内获得高关注度的原因之一。

上手建议：从 README 的 Quick Start 入手，先在单个项目上跑通捕获—压缩—注入的完整闭环，观察记忆质量；贡献方面可优先关注各 Agent 适配器和压缩策略模块，这两处是扩展性和效果的关键。注意项目星标数异常偏高（10 万+），建议结合实际代码活跃度和 issue 质量判断成熟度，避免被热度误导。

#2 TypeScript 连续 2 天

OpenMausBot

Open-source Grok Bot alternative with a virtual machine that bots can use

Stars **8,205** Forks **1,174** Today **+899**

https://github.com/milind-soni/OpenMausBot

项目简介：OpenMausBot 是一个开源的“AI 机器人聊天应用”，把每个 AI agent 包装成侧边栏里的一个联系人，像用 Telegram 一样与它们对话。它解决的是“单一助手塞进一个对话框”的形态问题——让用户拥有一支各自独立、可分工的 AI 团队，且完全本地优先、自带模型。

核心功能：每个 bot 拥有独立人格、独立模型、独立云电脑和已连接的应用；底层可运行 Claude 或 Codex 等真实 agent，用户可观察其工作过程并审批关键操作；支持“自带 agent”（bring-your-own-agent），复用已有模型；提供跨平台桌面客户端（macOS/Windows/Ubuntu）；聊天式交互，像管理联系人一样管理 bot。

适用场景：适合希望把 AI 从“问答工具”升级为“可并行协作的助手团队”的个人开发者、重度 AI 用户和小团队；典型场景包括多任务并行处理、需要审批环节的自动化工作流、以及注重数据本地化、不愿依赖闭源 SaaS 的用户。

技术亮点：采用 TypeScript（strict）+ React 19 + Electron 构建跨平台桌面应用，架构上把“聊天界面”与“本地运行的 agent 运行时”解耦；每个 bot 绑定独立虚拟机/云电脑，实现隔离与并行；本地优先设计让模型调用和数据处理留在用户侧，兼顾隐私与可控性。

上手建议：普通用户直接从 Releases 下载对应平台安装包（Mac 已签名公证）即可体验；开发者建议先读 README 中 “Why” 与架构部分，理解 bot 与 agent 运行时的关系，再从 issue 或 PR welcome 入手；注意项目与 xAI/Meta/OpenAI 无关联，也无任何代币，警惕同名加密货币骗局。

#3 Rust 连续 7 天

impeccable

The design vocabulary for your coding agent. Steer with polish and bolder, review with critique and audit, catch AI slop before it ships.

Stars **96,665** Forks **5,694** Today **+2,299**

https://github.com/pbakaus/impeccable

项目简介

Impeccable 是一套给 AI 编码助手用的「设计词汇表」，本质是一组结构化的斜杠命令（如 `/impeccable polish`、`/impeccable critique`），让开发者能用精确的指令引导 AI 做出更好的界面设计，而不是靠「让它更好看」这类模糊要求碰运气。它要解决的核心痛点是 AI 生成界面时反复出现的「AI 味」——Inter 字体、紫蓝渐变、卡片套卡片、彩色背景上的灰字等千篇一律的默认审美。

核心功能

- **Steer（引导）类命令**：`bolder`、`quieter`、`distill`、`colorize`、`typeset`、`layout` 等，针对单一设计动作加载专门 playbook，把模糊的审美诉求变成可执行的具体操作。
- **Review（审查）类命令**：`critique` 按可用性启发式打分并排序待修项，`audit` 检查无障碍、性能、主题和响应式，发现按 P0–P3 分级。
- **Slop 检测器**：62 条确定性规则识别「AI 味」特征和排版/间距等质量问题，纯规则实现、不调用 LLM、无需 API key。
- **上下文文件机制**：`init` 生成 `PRODUCT.md`（产品目标与约束），`document` 生成 `DESIGN.md`（现有视觉系统），后续命令都读取它们以保持一致性。
- **实时迭代**：`live` 可在浏览器中点选元素循环切换变体，`generate` 直接产出指定元素的多个变体。

适用场景

主要面向已经在用 Claude Code、Cursor、Codex CLI、Copilot 等 AI 编码工具的开发者，尤其是对界面质量有要求、但缺乏设计语言来精确指挥 AI 的前端/全栈工程师。最擅长的是**迭代已有界面**——指向某个页面或组件逐步打磨；也能用于新页面构建，但官方也承认那是更难的模式。

技术亮点

用 Rust 实现，意味着安装和运行轻量、跨平台、无运行时依赖；62 条检测规则是确定性的静态分析，不依赖模型推理，结果可复现且零成本。产品形态上它并不自己「生成设计」，而是作为一层**命令协议 + 上下文文件**注入到各家 agent 中，这种「不绑定具体模型、适配 20+ 工具」的设计让它具备很强的通用性——README 里同模型同需求下的前后对比图（默认奶油色 vs. 围绕一张夜景图构建整页）正是这套词汇表价值的直观体现。

上手建议

从项目根目录执行 `npx impeccable install`，重载你的编码工具后运行 `/impeccable init`（Codex 用 `$impeccable init`）建立项目上下文。建议先挑**单个组件**试水，如 `/impeccable critique the pricing cards`，窄目标效果远好于整页泛审；常用命令可用 `pin` 固化为快捷方式。想深入可查阅 impeccable.style/docs，贡献方面可从检测规则或命令 playbook 入手，这两部分是项目价值密度最高的地方。

#4 TypeScript 连续 2 天

OpenBot

Open-source AI coworkers that each get a computer of their own: a browser, files and tools, with every action decided before it happens and recorded after. Bring any AG-UI agent.

Stars **8,577** Forks **1,088** Today **+707**

https://github.com/CopilotKit/OpenBot

项目简介：OpenBot 是 CopilotKit 推出的开源 AI 同事平台，定位是"企业真正能拥有的 AI 助手"——形态类似 ChatGPT/Claude，但完全跑在自有基础设施上，代码可随意修改。它解决的核心问题是：企业想用 AI Agent 处理实际工作，却不愿把数据和工具控制权交给第三方 SaaS。

核心功能：每个 AI 同事拥有独立的浏览器（含自己的登录态）、文件系统和被授权的工具集，互不干扰；所有操作"事前决策、事后留痕"，通过统一网关做审批与审计；内置 General Assistant、Knowledge 等开箱即用的 Bot，并提供金融场景下十余个单职责 Bot 模板（读报销单、写发布说明、分诊工单等）；支持接入任意 AG-UI 协议 Agent，也可在 `/bots` 界面直接创建。

适用场景：适合对数据主权敏感、需要 AI 代理操作真实系统（浏览器登录、读写文件、调用内部 MCP 服务）的企业团队，尤其是金融、合规等强审计要求的行业。典型用法是把它当模板克隆下来，替换 `examples/` 下的租户配置，改造成自家产品形态的内部 AI 工作台。

技术亮点：以 AG-UI 开放协议作为 Agent 接入标准，框架无关，手写或任意框架的 Agent 都能"入职"；所有对计算机、文件、MCP、组件的操作收敛到单一网关，实现可审计的权限边界；Docker Compose 一键拉起全栈，数据落在自有 PostgreSQL，模型凭据加密存储且不入日志，模型本身不绑定。

上手建议：从 README 的 Quick start 开始，本地克隆后 `.env.example` 默认 `OPENBOT_SINGLE_USER=true`，无需配 OAuth 即可直接体验；熟悉后重点读 `examples/fintech/` 下的 Bot 目录结构，照葫芦画瓢写自己的同事。项目处于 Beta 且明确是"模板而非产品"，贡献前建议先看 `docs/README.md` 和 CI/安全流水线配置，Issue 和 PR 都欢迎。

#5 TypeScript NEW

ziwei-doushu

紫微斗数开源排盘引擎 — 基于倪海厦《天纪》体系，含完整排盘算法、四化系统、格局知识库、古籍原文数据

Stars **11,013** Forks **2,207** Today **+1,146**

https://github.com/Renhuai123/ziwei-doushu

项目简介：这是一个基于倪海厦《天纪》体系的紫微斗数开源排盘引擎，用 TypeScript 实现了完整的排盘算法、四化系统和格局知识库。它解决的核心问题是：把传统命理中原本封闭在商业软件里的排盘算法和解读数据公开出来，让开发者能自行构建排盘与解读应用。项目同时附带古籍原文和倪师讲课原话数据，强调“算法是公开的传统智慧，不应被锁在围墙里”。

核心功能：一是完整排盘引擎，支持真太阳时校正、中国夏令时判定、农历公历互转、星曜亮度（默认《斗数全书》）与流派设置；二是四化系统与格局知识库，含 13 维解读维度；三是发布 v4.0 解读数据集——67.5 万张命盘 × 13 维共 877 万段解读，按年份分 77 个文件打包下载；四是倪师《天纪》原话 1848 条语料库，每条注明集数与时间戳可回放；五是配套 Demo 界面与在线站点，展示排盘、AI 解读、命盘历史。

适用场景：适合想开发命理类 App 或网站的开发者直接复用排盘内核；适合研究者或学习者用公开数据集做解读文本分析、知识库检索；也适合对紫微斗数感兴趣、想核对排盘结果准确性的爱好者。由于数据按年份切分且体积可控（压缩后 161 MB），做离线命盘查询工具也很合适。

技术亮点：工程细节相当扎实——真太阳时同时处理经度差与均时差，夏令时只判定不自动扣除、把改钟表的决定权交给用户并可撤销，这种“引擎不替用户做有争议判断”的设计很克制。数据侧用 77 个文件逐字节比对验证一致性，README 里对每处断语标注出处、删除查无实据的引文、对“两头说”的断语按盘只显示成立的一种，体现了对数据质量的认真态度。依赖从 483 个精简到 185 个并清理安全告警，也说明维护者在持续治理。

上手建议：先读“快速开始”跑通本地排盘，再对照“排盘引擎”章节了解模块划分（algorithm、true-solar-time、dst-cn、lunar-solar 等）；想用数据就直接下 Release v4.0 里需要的年份文件，注意 v4.0 是按旧默认亮度生成的。贡献方面，README 明确了开源边界（断语库不开源），可以先从补城市经度、修 bug、完善文档这类 issue/PR 入手，遵循仓库的贡献指南和问题模板。

#6 Shell NEW

skills

A collection of AI agent skills for working with Expo projects and Expo Application Services

Stars 3,821 Forks 243 Today +233

https://github.com/expo/skills

项目简介：这是 Expo 官方团队推出的 AI Agent 技能库，本质上是一套结构化的"知识包"，让 Claude Code、Cursor 等 AI 编程助手能够准确理解 Expo 生态的 API 用法、工作流和平台约束。它解决的核心痛点是：通用 AI 模型对 Expo/EAS 这类垂直框架的细节了解不足，容易给出过时或错误的建议。

核心功能： - 提供按任务场景拆分的技能模块（如 `expo-router`、`expo-animation`、`expo-data-fetching`），Agent 根据上下文自动选用合适技能 - 覆盖从项目结构、原生 UI、设计系统到 EAS 构建部署、SDK 升级、调试的完整链路 - 通过官方插件市场（Claude Code / Codex）或 skills CLI 一键安装，支持多 Agent 平台 - 明确区分免费开源框架技能与付费 EAS 服务技能，服务类技能自带成本/套餐限制说明

适用场景：主要面向使用 Expo 开发 React Native 应用的团队和独立开发者，尤其是已经在用 AI 编程助手写代码、但苦于模型对 Expo 细节"幻觉"较多的人。典型场景包括搭建 Expo Router 导航结构、编写 EAS 预览构建工作流、执行 SDK 版本升级、排查 EAS Update 发布健康度等。

技术亮点：项目本身用 Shell 组织，实质是"提示工程 + 文档工程"的产物——每个技能以 `SKILL.md` 为载体，把官方文档、CLI 行为、iOS/Android 平台差异压缩成 Agent 可直接消费的上下文。它刻意保持"文档和 CLI 才是事实来源"的定位，技能只负责帮 Agent 正确应用，避免知识库与真实工具脱节。这种"官方维护、按需路由"的设计比单一大 prompt 更可维护。

上手建议：从 `expo-overview` 技能入手，理解它如何路由到具体技能；安装直接用 `npx skills@latest add expo/skills --skill '*'`，然后重启 Agent 会话让它发现 `SKILL.md`。想贡献的话，建议先读现有技能的写法规范，从补充某个具体 API 的边界情况或平台约束入手，保持"简洁、可路由、不替代官方文档"的原则。

#7 Rust NEW

macro

Macro is a unified workspace for teams: email, chat, docs, tasks, agents, calls, and CRM — @-linked together with shared AI memory.

Stars **5,756** Forks **537** Today **+117**

https://github.com/macro-inc/macro

项目简介：Macro 是一个面向小型团队的“一体化工作操作系统”，试图把邮件、聊天、文档、任务、CRM、AI 智能体等原本分散在 Slack、Notion、Linear、HubSpot、Superhuman 中的能力整合进同一个界面。它解决的核心痛点是：团队规模扩大后工具各自为政，公司变得“不可计算”、靠 MCP 和 Zapier 勉强拼接的混乱状态。

核心功能：统一多账户收件箱（邮件+消息+@提及+任务混合 triage，键盘优先操作）；Linear 风格的任务系统与频道、邮件、智能体深度打通；基于 CRDT 的实时协作 Markdown 文档；带团队级共享记忆、可代为执行操作的 AI Agents；以及记录转写并沉淀到团队记忆的通话与 CRM 客户对象管理。

适用场景：适合 10–50 人左右、希望用一套系统替代多套 SaaS 的初创团队或大公司内的小团队，尤其是工程驱动型团队——他们把邮件、讨论、任务、客户信息都放在一个可 @链接、可搜索的空间里，让人类和 AI 智能体共享同一份上下文。

技术亮点：前端用 SolidJS、后端用 Rust，主打速度与可靠性；所有表面共享同一后端，跨文档/任务/消息/邮件的引用以**双向图**原生存储，而非靠外部集成拼接；文档基于 CRDT 实现实时协作；AI 层设计了专门的 tools/MCP 接口（如统一搜索可直接检索邮件附件解析出的 PDF），让智能体更精准地取数。

上手建议：普通用户可从官网注册或预约 demo 直接体验产品；开发者想贡献代码应先读 CONTRIBUTING.md，并熟悉 Rust 后端与 SolidJS 前端的架构；建议先跑通本地环境、从文档中挑选一个具体 block（如邮件或任务）理解其数据模型和双向图设计，再着手小范围改动。

#8 Shell NEW

skills

Skills for Real Engineers. Straight from my .agents directory.

Stars **307,201** Forks **25,849** Today **+882**

https://github.com/mattpocock/skills

项目简介：这是 Matt Pocock（TypeScript 社区知名教育者）整理的一套 AI 编程代理「技能包」，直译自他日常在 `.agents` 目录里实际使用的配置。它要解决的核心问题是：当前 GSD、BMAD、Spec-Kit 这类流程框架虽然帮你管流程，但也剥夺了控制权、让流程本身的 bug 难以排查——他反其道而行，提供小而可组合、可随意改造的技能。

核心功能：最核心的是 `/grill-me` 和 `/grill-with-docs` 两个「拷问」技能，在动手前让 agent 反复追问你的真实需求，解决人与 AI 之间的沟通错位；配套的 `/triage` 按你自定义的标签体系处理工单；`/setup-matt-pocock-skills` 一次性配置好 issue tracker、标签和文档存放位置。整体按「工程类」和「效率类」两类组织，可跨模型使用。

适用场景：主要面向用 Claude Code、Codex、GitHub Copilot、Gemini CLI 等编码代理做真实工程（而非「vibe coding」）的开发者。适合那些觉得重型流程框架太束缚、又希望 agent 在动手前真正理解需求的人，尤其适合需求模糊、容易返工的项目。

技术亮点：项目本身以 Shell 为主，本质是纯 Markdown 技能定义 + 多代理安装脚本，几乎零运行时依赖。设计哲学是「小而可组合、易改造」，不绑定单一模型；同时通过插件市场（Claude/Codex/Copilot）或 `npx skills` 分发，兼顾自动更新与手动可编辑两种模式。

上手建议：先挑一个你常用的代理，按 README 的 30 秒安装步骤装好，然后每个仓库跑一次 `/setup-matt-pocock-skills` 完成配置。建议从 `/grill-me` 开始体验其核心价值，再按需挑选其他技能；想贡献或定制，直接读各 `SKILL.md` 源文件改即可——作者本人就鼓励你「hack around with them, make them your own」。

#9 TypeScript NEW

t3code

Stars **32,481** Forks **8,597** Today **+920**

https://github.com/pingdotgg/t3code

项目简介：T3 Code 是一个"智能体控制面板"（agent harness control surface），由知名开发者 Theo（pingdotgg）团队打造。它本身不提供 AI 能力，而是作为统一入口，让你在手机、网页或桌面端远程操控本机已安装的各类编码智能体（Claude Code、Codex、Cursor 等），解决"多智能体分散、无法远程统一管理"的痛点。

核心功能：一是多智能体统一接入，兼容 Claude Code、Codex、Cursor、Grok Build、OpenCode、Antigravity 六种主流 CLI 智能体；二是全平台覆盖，提供 iOS/Android App、Web 应用和 Electron 桌面端，支持手机远程访问本机任务；三是后台服务化运行（`t3 service install`），可常驻后台持续调度；四是支持多账号与 MCP 协议连接外部智能体；五是内置权限模式、快捷键、源码控制集成等开发体验细节。

适用场景：适合重度使用编码智能体的开发者——尤其是希望在离开电脑时用手机查看/推进本机 agent 任务、或同时管理多个不同厂商智能体的人。也适合想统一工作流、避免在多个 CLI 工具间来回切换的团队和个人。

技术亮点：核心卖点在于"远程优先"架构——本机跑服务端、多端做控制面，把本地算力和远程操控解耦；宣称高性能且完全开源，明确鼓励用户 fork 自建。构建工具链采用 Vite+（vp 命令），体现对现代前端工程化的追求。值得注意的是它刻意"薄"——不重新实现 agent，而是做编排与交互层，这与 Cursor、Claude Desktop 等"全家桶"路线形成差异化。

上手建议：先按 README 安装并登录至少一个受支持的 CLI 智能体，然后 `curl` 安装脚本或 `npx t3@latest` 快速试用，桌面端也可通过 Homebrew/winget/scoop 等包管理器安装。项目尚处早期、明确表示暂不接受大型贡献，想参与应先读 `CONTRIBUTING.md`，从小的 bug 修复入手，并在 Ideas discussion 提需求。

#10 Python 连续 4 天

DeepTutor

DeepTutor: Lifelong Personalized Tutoring.https://deeptutor.info/.

Stars **51,509** Forks **6,308** Today **+2,282**

https://github.com/HKUDS/DeepTutor

项目简介：DeepTutor 是香港大学数据智能实验室（HKUDS）推出的"终身个性化辅导"开源项目，试图用 AI 为每个学习者构建一个长期陪伴、持续理解其知识状态与学习目标的私人导师，解决传统 AI 教育工具"一次性问答、缺乏记忆与个性化"的问题。项目以 Python 为主、前端用 Next.js，采用 Apache 2.0 协议，并配有 arXiv 论文，属于研究驱动型的开源产品。

核心功能：一是"终身个性化"——通过长期记忆学习者的知识水平、薄弱点和偏好，让辅导随学随进；二是多模态/多语言支持，README 提供十余种语言版本，面向全球用户；三是 Agent 化交互，README 提到"Agent-native interface"与 CLI，说明辅导能力可通过命令行/智能体方式调用与编排；四是生态化设计，提出 EduHub / Skills 社区概念，允许扩展教学技能与教学资源；五是配套完整的文档站（deeptutor.info）与协作入口。

适用场景：适合自学者、学生做长期学科辅导与知识查漏补缺；也适合教师/教研团队作为个性化教学辅助工具；对 AI 教育研究者而言，它是研究"长期记忆 + 个性化教学 Agent"的可复现实验平台；开发者则可基于其 Skills 生态构建垂直学科辅导应用。

技术亮点：Python 3.11+ 后端搭配 Next.js 16 前端，属于典型的"AI Agent + 现代 Web 应用"架构；强调"终身"意味着系统需要跨会话的持久化记忆与学习者建模，这是区别于普通 RAG 问答的关键；CLI 与 Agent-native 接口的设计表明其定位不只是聊天界面，而是可被程序化调用的辅导引擎；背靠 HKUDS 且有论文支撑，工程与学术结合度较高。

上手建议：先访问 deeptutor.info 文档站按 Get Started 跑通本地部署，体验个性化辅导流程；想深入可阅读其 arXiv 论文理解记忆与个性化机制；贡献方面可从 GitHub Issues 中的 Roadmap 投票或提新需求入手，并按 CONTRIBUTING.md 的分支策略与代码规范提交 PR，社区有 Discord、飞书、微信群可直接交流。

数据来源：github.com/trending

报告日期：2026-10-30

由 DeepSeek AI 提供智能分析 | 自动生成